

Sınıf tabanlı görünüm

Bir view (görünüm), bir isteği alan ve bir yanıt döndüren çağrılabilir bir yapıdır. Bu, sadece bir fonksiyondan daha fazlası olabilir ve Django, view olarak kullanılacak bazı Class örnekleri sunar. Bunlar; kalıtım ve mixin özelliklerinden yararlanarak görünümlerinizi yapılandırmanıza ve kodu yeniden kullanmanıza olanak tanır. Ayrıca, ileride değineceğimiz bazı temel görevler için jenerik görünüm de mevcuttur; ancak kendi kullanım durumunuza uygun, yeniden kullanılabilir görünümlerden oluşan kendi yapınızı tasarlamak isteyebilirsiniz. Tüm detaylar için Class-tabanlı görünüm referans dokümantasyonuna bakınız.

- Sınıf tabanlı görünümlere giriş
- Yerleşik sınıf tabanlı genel görünüm
- Sınıf tabanlı görünümle form işleme
- Sınıf tabanlı görünümle mixin kullanımı

Temel örnekler

Django, çok çeşitli uygulamalara uygun temel görünüm sınıfları sunar. Tüm görünüm, görünümü URL'lere bağlama, HTTP yönteminin yönlendirilmesi ve diğer yaygın özellikleri yöneten View sınıfından miras alır. RedirectView, HTTP yönlendirmesi sağlar; TemplateView ise şablon oluşturma işlevini de yerine getirebilmesi için temel sınıfı genişletir.

URLconf'unuzda kullanım

Genel görünümü kullanmanın en doğrudan yolu, bunları doğrudan URLconf dosyanızda oluşturmaktır. Sınıf tabanlı bir görünümde yalnızca birkaç özneliği değiştiriyorsanız, bunları doğrudan `as_view()` yönteminin çağrısına aktarabilirsiniz:

```
from django.urls import path
from django.views.generic import TemplateView

urlpatterns = [
    path("about/", TemplateView.as_view(template_name="about.html")),
]
```

`as_view()` işlevine aktarılan tüm argümanlar, sınıfta ayarlanan özelliklerin yerine geçer. Bu örnekte, `TemplateView` sınıfında `template_name` özelliğini ayarladık. Benzer bir geçersiz kılma yöntemi, `RedirectView` sınıfındaki `url` özelliği için de kullanılabilir.

Genel görünümün alt sınıfları

Genel görünümü kullanmanın ikinci ve daha etkili yolu, mevcut bir görünümünden miras almak ve alt sınıfınızda yeni değerler veya yöntemler sağlamak için öznelikleri (örneğin `template_name`) veya yöntemleri (örneğin `get_context_data`) geçersiz kılmaktır. Örneğin, sadece `about.html` adlı tek bir şablonu görüntüleyen bir görünümü ele alalım. Django'da bunu yapmak için `TemplateView` adlı bir genel görünüm mevcuttur; bu yüzden bu sınıfın alt sınıfını oluşturabilir ve şablon adını geçersiz kılabiliriz:

```
# some_app/views.py
from django.views.generic import TemplateView

class AboutView(TemplateView):
    template_name = "about.html"
```

Ardından bu yeni görünümü URLconf dosyamıza eklememiz gerekiyor. `TemplateView` bir işlev değil, bir sınıftır; bu nedenle URL'yi, sınıf tabanlı görünüm için işlev benzeri bir erişim sağlayan `as_view()` sınıf yöntemine yönlendiriyoruz:

```
# urls.py
from django.urls import path
from some_app.views import AboutView

urlpatterns = [
    path("about/", AboutView.as_view()),
]
```

Yerleşik genel görünümün nasıl kullanıldığına dair daha fazla bilgi için, genel sınıf tabanlı görünümle ilgili bir sonraki konuya bakın.

Diğer HTTP yöntemlerini destekleme

Diyelim ki birisi, görünümü bir API olarak kullanarak HTTP üzerinden kitap kütüphanemize erişmek istiyor. API istemcisi ara sıra bağlanarak, son ziyaretinden bu yana yayınlanan kitapların verilerini indirecektir. Ancak o zamandan beri

yeni kitap yayınlanmamışsa, kitapları veritabanından almak, tam bir yanıt oluşturmak ve bunu istemciye göndermek CPU süresi ve bant genişliği israfı anlamına gelir. API'ye en son kitabın ne zaman yayınlandığını sormak daha uygun olabilir.

URLconf dosyasında kitap listesi görünümüne giden URL'yi eşliyoruz:

```
from django.urls import path
from books.views import BookListView

urlpatterns = [
    path("books/", BookListView.as_view()),
]
```

Ve görünüm:

```
from django.http import HttpResponse
from django.views.generic import ListView
from books.models import Book

class BookListView(ListView):
    model = Book

    def head(self, *args, **kwargs):
        last_book = self.get_queryset().latest("publication_date")
        response = HttpResponse(
            # RFC 1123 tarih biçimi.
            headers={
                "Last-Modified": last_book.publication_date.strftime(
                    "%a, %d %b %Y %H:%M:%S GMT"
                )
            },
        )
        return response
```

Görünümüne bir GET isteği ile erişilirse, yanıtta bir nesne listesi döndürülür (book_list.html şablonu kullanılarak). Ancak istemci bir HEAD isteği gönderirse, yanıtın gövdesi boştur ve Last-Modified başlığı en son kitabın ne zaman yayınlandığını gösterir. Bu bilgilere dayanarak, istemci tam nesne listesini indirebilir veya indirmeyebilir.

Asenkron sınıf tabanlı görünüm

Daha önce gösterilen senkron (def) yöntem işleyicilerinin yanı sıra, View alt sınıfları await kullanarak asenkron koddan yararlanmak üzere asenkron (async def) yöntem işleyicileri tanımlayabilir:

```
import asyncio
from django.http import HttpResponse
from django.views import View

class AsyncView(View):
    async def get(self, request, *args, **kwargs):
        # await ile G/Ç işlemleri gerçekleştirin.
        await asyncio.sleep(1)
        return HttpResponse("Merhaba, asenkron dünya!")
```

Aynı görünüm sınıfı içinde, kullanıcı tanımlı tüm yöntem işleyicileri ya def kullanılarak senkronize olmalı ya da async def kullanılarak tamamen asenkron olmalıdır. def ve async def bildirimleri karıştırılırsa, as_view() işlevinde bir ImproperlyConfigured istisnası atılır.

Django, eşzamansız görünümeleri otomatik olarak algılar ve bunları eşzamansız bir bağlamda çalıştırır. Django'nun eşzamansız desteği ve eşzamansız görünümelerin en iyi şekilde nasıl kullanılacağı hakkında daha fazla bilgiyi Eşzamansız destek bölümünde bulabilirsiniz.

Sınıf tabanlı görünümlere giriş

Sınıf tabanlı görünüm, görünümleri işlevler yerine Python nesneleri olarak uygulamak için alternatif bir yol sunar. Bunlar işlev tabanlı görünümlerin yerini almaz, ancak işlev tabanlı görünümlerle karşılaştırıldığında bazı farklılıklar ve avantajlar sunar:

- Belirli HTTP yöntemleriyle (GET, POST vb.) ilgili kodun düzenlenmesi, koşullu dallanma yerine ayrı yöntemler kullanılarak sağlanabilir.

- Mixinler (çoklu miras) gibi nesne yönelimli teknikler, kodu yeniden kullanılabilir bileşenlere ayırmak için kullanılabilir.

Genel görünüm, sınıfa dayalı görünüm ve sınıfa dayalı genel görünümün ilişkisi ve geçmişi

Başlangıçta sadece görünüm işlevi sözleşmesi vardı; Django, işlevinize bir `HttpRequest` aktarıyor ve karşılığında bir `HttpResponse` bekliyordu. Django'nun sağladığı destek bu kadarla sınırlıydı.

Görünüm geliştirmede yaygın olarak kullanılan kalıplar ve deyimlerin olduğu, çok erken bir aşamada fark edildi. Bu kalıpları soyutlamak ve yaygın durumlarda görünüm geliştirmeyi kolaylaştırmak amacıyla işlev tabanlı genel görünüm ortaya çıktı.

İşlev tabanlı genel görünümün sorunu, basit durumları iyi bir şekilde kapsıyor olsalar da bazı yapılandırma seçeneklerinin ötesinde genişletilememeleri veya özelleştirilememeleriydi; bu da birçok gerçek dünya uygulamasında kullanışlılıklarını sınırlıyordu.

Sınıf tabanlı genel görünüm, görünüm geliştirmeyi kolaylaştırmak amacıyla işlev tabanlı genel görünümle aynı hedefle oluşturuldu. Ancak, çözümün mixin'lerin kullanımıyla uygulanma şekli, sınıf tabanlı genel görünümün işlev tabanlı muadillerine göre daha genişletilebilir ve esnek olmasını sağlayan bir araç seti sunar.

Geçmişte işlev tabanlı genel görünümü denediyseniz ve bunların yetersiz olduğunu düşündüyseniz, sınıf tabanlı genel görünümü işlev tabanlı görünümün sınıf tabanlı eşdeğeri olarak değil, genel görünümün çözmesi amaçlanan orijinal sorunları çözmek için yeni bir yaklaşım olarak düşünmelisiniz.

Django'nun sınıf tabanlı genel görünüm oluşturmak için kullandığı temel sınıflar ve mixinlerden oluşan araç seti, maksimum esneklik sağlayacak şekilde tasarlanmıştır; bu nedenle, en basit kullanım senaryolarında muhtemelen ilgilenmeyeceğiniz varsayılan yöntem uygulamaları ve öznitelikler şeklinde birçok kanca içerir. Örneğin, `form_class` için sizi sınıf tabanlı bir öznitelikle sınırlamak yerine, uygulama bir `get_form` yöntemi kullanır; bu yöntem, varsayılan uygulamasında sınıfın `form_class` özniteliğini döndüren bir `get_form_class` yöntemini çağırır. Bu, hangi formun kullanılacağını belirtmek için bir öznitelikten tamamen dinamik, çağrılabilir bir kancaya kadar çeşitli seçenekler sunar. Bu seçenekler basit durumlar için gereksiz bir karmaşıklık yaratıyor gibi görünebilir, ancak bunlar olmadan daha gelişmiş tasarımlar sınırlı kalırdı.

Sınıf tabanlı görünümün kullanımı

Temelde, sınıf tabanlı bir görünüm, tek bir görünüm işlevi içindeki koşullu dallanma kodları yerine, farklı HTTP istek yöntemlerine farklı sınıf örnek yöntemleriyle yanıt vermenizi sağlar.

Dolayısıyla, bir görünüm işlevinde HTTP GET isteğini işleyen kod şuna benzer bir şekilde görünür:

```
from django.http import HttpResponse

def my_view(request):
    if request.method == "GET":
        # <view mantığı>
        return HttpResponse("result")
```

Sınıf tabanlı bir yaklaşımda bu şu şekilde olur:

```
from django.http import HttpResponse
from django.views import View

class MyView(View):
    def get(self, request):
        # <view mantığı>
        return HttpResponse("result")
```

Django'nun URL çözümleyicisi, isteği ve ilgili argümanları bir sınıfa değil, çağrılabilir bir işlevine göndermeyi beklediğinden, sınıf tabanlı görünüm, ilişkili desene uyan bir URL için istek geldiğinde çağrılacak bir işlev döndüren `as_view()` sınıf yöntemine sahiptir. Bu işlev, sınıfın bir örneğini oluşturur, özelliklerini başlatmak için `setup()` işlevini çağırır ve ardından `dispatch()` yöntemini çağırır. `dispatch`, isteğin GET, POST vb. olup olmadığını belirlemek için isteğe bakar ve tanımlanmış istek eşleşen bir yönteme iletir, yoksa `HttpResponseNotAllowed` hatası verir:

```
# urls.py
from django.urls import path
from myapp.views import MyView

urlpatterns = [
    path("about/", MyView.as_view()),
]
```

Yönteminizin döndürdüğü sonucun, işlev tabanlı bir görünümünden döndürdüğünüz sonuçla, yani bir tür `HttpResponse` nesnesiyle aynı olduğunu belirtmek gerekir. Bu, sınıf tabanlı bir görünüm içinde `http` kısayollarının veya `TemplateResponse` nesnelerinin kullanılmasının geçerli olduğu anlamına gelir.

Minimal sınıf tabanlı bir yaklaşım, işlevini yerine getirmek için herhangi bir sınıf özneliğine ihtiyaç duymasa da sınıf öznelikleri birçok sınıf tabanlı tasarımda yararlıdır ve sınıf özneliklerini yapılandırmanın veya ayarlamanın iki yolu vardır.

Bunlardan ilki, alt sınıf oluşturma ve alt sınıftaki özneliklerle yöntemleri geçersiz kılma konusunda Python'un standart yöntemidir. Yani, üst sınıfınızda şöyle bir `greeting` özneliği varsa:

```
from django.http import HttpResponse
from django.views import View

class GreetingView(View):
    greeting = "Good Day"

    def get(self, request):
        return HttpResponse(self.greeting)
```

Bunu bir alt sınıfta geçersiz kılabilirsiniz:

```
class MorningGreetingView(GreetingView):
    greeting = "Morning to ya"
```

Başka bir seçenek de sınıf özneliklerini `URLconf` içindeki `as_view()` çağırısına anahtar kelime argümanları olarak yapılandırmaktır:

```
urlpatterns = [
    path("about/", GreetingView.as_view(greeting="G'day")),
]
```

Not

Sınıfınız kendisine yönlendirilen her istek için bir örnek oluşturulurken, `as_view()` giriş noktası aracılığıyla ayarlanan sınıf öznelikleri yalnızca URL'leriniz içe aktarıldığı anda bir kez yapılandırılır.

Mixinlerin kullanımı

Mixinler, birden fazla üst sınıfın davranış ve özelliklerinin birleştirilebildiği bir tür çoklu miras biçimidir.

Örneğin, genel sınıf tabanlı görünümelerde, temel amacı `render_to_response()` yöntemini tanımlamak olan `TemplateResponseMixin` adlı bir mixin bulunmaktadır. `View` temel sınıfının davranışıyla birleştirildiğinde, sonuç, istekleri uygun eşleşen yöntemlere (`View` temel sınıfında tanımlanan bir davranış) yönlendirecek ve `template_name` özneliğini kullanarak bir `TemplateResponse` nesnesi (`TemplateResponseMixin`'de tanımlanan bir davranış) döndüren bir `render_to_response()` yöntemine sahip bir `TemplateView` sınıfıdır.

Mixin'ler, birden fazla sınıfta kodu yeniden kullanmanın mükemmel bir yoludur, ancak bazı maliyetleri de vardır. Kodunuz mixin'ler arasında ne kadar dağınık olursa, alt sınıfı okumak ve tam olarak ne yaptığını anlamak o kadar zorlaşır ve derin bir miras ağacına sahip bir şeyin alt sınıfını oluşturuyorsanız, hangi mixin'lerden hangi yöntemleri geçersiz kılmanız gerektiğini bilmek o kadar zorlaşır.

Ayrıca, yalnızca tek bir genel görünümünden miras alınabileceğini de unutmayın; yani, yalnızca bir üst sınıf `View`'dan miras alabilir ve geri kalanlar (varsa) mixin olmalıdır. `View`'dan miras alan birden fazla sınıftan miras almaya çalışmak — örneğin, bir listenin en üstünde bir form kullanmaya çalışmak ve `ProcessFormView` ile `ListView`'u birleştirmek — beklendiği gibi çalışmayacaktır.

Sınıf tabanlı görünümlemlerle formların işlenmesi

Formları işleyen temel bir işlev tabanlı görünüm şuna benzer olabilir:

```
from django.http import HttpResponseRedirect
from django.shortcuts import render

from .forms import MyForm

def myview(request):
    if request.method == "POST":
        form = MyForm(request.POST)
        if form.is_valid():
            # <form temizlenmiş verilerini işle>
            return HttpResponseRedirect("/success/")
    else:
```

```

        form = MyForm(initial={"key": "value"})

    return render(request, "form_template.html", {"form": form})

```

Benzer bir sınıf tabanlı görünüm şöyle görünebilir:

```

from django.http import HttpResponseRedirect
from django.shortcuts import render
from django.views import View

from .forms import MyForm

class MyFormView(View):
    form_class = MyForm
    initial = {"key": "value"}
    template_name = "form_template.html"

    def get(self, request, *args, **kwargs):
        form = self.form_class(initial=self.initial)
        return render(request, self.template_name, {"form": form})

    def post(self, request, *args, **kwargs):
        form = self.form_class(request.POST)
        if form.is_valid():
            # <form temizlenmiş verilerini işle>
            return HttpResponseRedirect("/success/")

        return render(request, self.template_name, {"form": form})

```

Bu basit bir örnektir, ancak URLconf yapılandırması yoluyla `form_class` gibi sınıf özniteliklerini geçersiz kılıp bu görünümü özelleştirme ya da alt sınıf oluşturup bir veya daha fazla yöntemi (ya da her ikisini birden!) geçersiz kılma seçeneğinizin olduğunu görebilirsiniz.

Sınıf tabanlı görünümeleri süsleme

Sınıf tabanlı görünümelerin genişletilmesi, sadece `mixins`'lerin kullanımıyla sınırlı değildir. Dekoratörleri de kullanabilirsiniz. Sınıf tabanlı görünümeler işlev olmadığından, bunları dekore etme işlemi, `as_view()` işlevini kullanıp kullanmadığınıza veya bir alt sınıf oluşturup oluşturmadığınıza bağlı olarak farklı şekilde gerçekleşir.

URLconf'ta biçimlendirme

`as_view()` yönteminin çıktısını süsleyerek sınıf tabanlı görünümeleri özelleştirebilirsiniz. Bunu yapmanın en kolay yolu, görünümü uyguladığınız URLconf dosyasıdır:

```

from django.contrib.auth.decorators import login_required, permission_required
from django.views.generic import TemplateView

from .views import VoteView

urlpatterns = [
    path("about/",
        login_required(TemplateView.as_view(template_name="secret.html"))),
    path("vote/", permission_required("polls.can_vote")(VoteView.as_view())),
]

```

Bu yaklaşım, dekoratörü her bir örnek için ayrı ayrı uygular. Bir görünümün tüm örneklerinin dekorasyona tabi tutulmasını istiyorsanız, farklı bir yaklaşım benimsemeniz gerekir.

Sınıfın dekorasyonu

Sınıf tabanlı görünümün her örneğini süslemek için, sınıf tanımının kendisini süslemeniz gerekir. Bunu yapmak için, süsleyiciyi sınıfın `dispatch()` yöntemine uygularsınız.

Bir sınıftaki yöntem, bağımsız bir işlevle tam olarak aynı değildir, bu nedenle bir işlev süsleyicisini doğrudan yöntemle uygulayamazsınız; önce onu bir yöntem süsleyicisine dönüştürmeniz gerekir. `method_decorator` süsleyicisi, bir işlev süsleyicisini yöntem süsleyicisine dönüştürür, böylece örnek yöntem üzerinde kullanılabilir. Örneğin:

```

from django.contrib.auth.decorators import login_required
from django.utils.decorators import method_decorator
from django.views.generic import TemplateView

class ProtectedView(TemplateView):

```

```
template_name = "secret.html"

@method_decorator(login_required)
def dispatch(self, *args, **kwargs):
    return super().dispatch(*args, **kwargs)
```

Ya da daha kısaca, sınıfı süsleyebilir ve süslenecek yöntemin adını name anahtar sözcüğü argümanı olarak iletebilirsiniz:

```
@method_decorator(login_required, name="dispatch")
class ProtectedView(TemplateView):
    template_name = "secret.html"
```

Birkaç yerde kullanılan ortak dekoratörleriniz varsa, bir dekoratör listesi veya tuple'ı tanımlayabilir ve method_decorator() işlevini defalarca çağırmak yerine bunu kullanabilirsiniz. Bu iki sınıf birbirinin aynısıdır:

```
decorators = [never_cache, login_required]

@method_decorator(decorators, name="dispatch")
class ProtectedView(TemplateView):
    template_name = "secret.html"

@method_decorator(never_cache, name="dispatch")
@method_decorator(login_required, name="dispatch")
class ProtectedView(TemplateView):
    template_name = "secret.html"
```

Dekoratörler, istekleri kendilerine aktarıldıkları sırayla işler. Örnekte, never_cache() işlevi login_required() işlevinden önce isteği işleyecektir.

Bu örnekte, ProtectedView sınıfının her örneği oturum açma korumasına sahip olacaktır. Bu örneklerde login_required işlevi kullanılmaktadır; ancak aynı davranış, LoginRequiredMixin kullanılarak da elde edilebilir.

Not

method_decorator, *args ve **kwargs parametrelerini sınıftaki süslenen yönteme aktarır. Yönteminiz uyumlu bir parametre kümesi kabul etmiyorsa, bir TypeError istisnası oluşturur.

Yerleşik sınıf tabanlı genel görünümler

Web uygulamaları yazmak, belirli kalıpları defalarca tekrarladığımız için monoton bir iş olabilir. Django, model ve şablon katmanlarında bu monotonluğun bir kısmını ortadan kaldırmaya çalışır, ancak web geliştiricileri bu sıkıcılığı görünüm düzeyinde de yaşarlar.

Django'nun genel görünümüleri, bu sıkıntıyı gidermek için geliştirilmiştir. Görünüm geliştirmede sıkça rastlanan belirli kalıpları ve yapıları alır ve bunları soyutlaştırır; böylece çok fazla kod yazmak zorunda kalmadan verilerin genel görünümelerini hızla yazabilirsiniz.

Nesnelerin bir listesini görüntülemek gibi belirli yaygın görevleri tanıyabilir ve herhangi bir nesnenin listesini görüntüleyen kod yazabiliriz. Ardından söz konusu model, URLconf'a ek bir argüman olarak aktarılabilir.

Django, aşağıdakileri yapmak için genel görünümle birlikte gelir:

- Tek bir nesneye ait liste ve ayrıntı sayfalarını görüntüleyin. Örneğin, konferansları yönetmek için bir uygulama geliştiriyor olsaydık, TalkListView ve RegisteredUserListView liste görünümlerine örnek olarak verilebilir. Tek bir konuşma sayfası ise "ayrıntı" görünümü olarak adlandırdığımız şeye bir örnektir.
- Tarih tabanlı nesneleri yıl/ay/gün arşiv sayfalarında, ilişkili ayrıntılarda ve "en son" sayfalarında sunun.
- Kullanıcıların yetkilendirme olsun ya da olmasın nesneleri oluşturmaya, güncellemesine ve silmesine izin verin.

Bu arayüzler bir araya geldiğinde, geliştiricilerin karşılaştığı en yaygın görevleri yerine getirmelerine olanak tanır.

Genel görünümünün genişletilmesi

Genel görünümünün kullanımıyla geliştirme sürecinin önemli ölçüde hızlanacağına şüphe yok. Ancak çoğu projede, genel görünümünün artık yetersiz kaldığı bir an gelir. Nitekim, yeni Django geliştiricilerinin en sık sorduğu soru, genel görünümünün daha geniş bir yelpazedeki durumları nasıl ele alabileceğidir.

Bu, genel görünümünün 1.3 sürümü için yeniden tasarlanmasının nedenlerinden biridir. Önceden, bunlar kafa karıştırıcı bir dizi seçeneğe sahip görünüm işlevleriydi; artık, URLconf'ta büyük miktarda yapılandırma aktarmak yerine, genel

görünümleri genişletmek için önerilen yol, bunların alt sınıflarını oluşturmak ve öz niteliklerini veya yöntemlerini geçersiz kılmaktır.

Bununla birlikte, genel görünümün bir sınırı olacaktır. Görünümünüzü bir genel görünümün alt sınıfı olarak uygulamakta zorlanıyorsanız, kendi sınıf tabanlı veya işlevsel görünümlerinizi kullanarak sadece ihtiyacınız olan kodu yazmanın daha etkili olduğunu görebilirsiniz.

Genel görünümün daha fazla örneği bazı üçüncü taraf uygulamalarda mevcuttur veya gerektiğinde kendi örneklerinizi yazabilirsiniz.

Nesnelerin genel görünümüleri

TemplateView elbette kullanışlıdır, ancak veri tabanı içeriğinizin görünümünü sunma söz konusu olduğunda Django'nun genel görünümüleri gerçekten öne çıkar. Bu çok yaygın bir görev olduğu için, Django nesnelerin liste ve ayrıntı görünümünü oluşturmaya yardımcı olacak birkaç yerleşik genel görünümle birlikte gelir.

Öncelikle, nesnelerin listesini veya tek bir nesneyi gösteren bazı örnekleri inceleyelim.

Şu modelleri kullanacağız:

```
# models.py
from django.db import models

class Publisher(models.Model):
    name = models.CharField(max_length=30)
    address = models.CharField(max_length=50)
    city = models.CharField(max_length=60)
    state_province = models.CharField(max_length=30)
    country = models.CharField(max_length=50)
    website = models.URLField()

    class Meta:
        ordering = ["-name"]

    def __str__(self):
        return self.name

class Author(models.Model):
    salutation = models.CharField(max_length=10)
    name = models.CharField(max_length=200)
    email = models.EmailField()
    headshot = models.ImageField(upload_to="author_headshots")

    def __str__(self):
        return self.name

class Book(models.Model):
    title = models.CharField(max_length=100)
    authors = models.ManyToManyField("Author")
    publisher = models.ForeignKey(Publisher, on_delete=models.CASCADE)
    publication_date = models.DateField()
```

Şimdi bir görünüm tanımlamamız gerekiyor:

```
# views.py
from django.views.generic import ListView
from books.models import Publisher

class PublisherListView(ListView):
    model = Publisher
```

Son olarak bu görünümü URL'lerinize ekleyin:

```
# urls.py
from django.urls import path
from books.views import PublisherListView

urlpatterns = [
    path("publishers/", PublisherListView.as_view()),
```

]

Yazmamız gereken Python kodu bu kadar. Ancak yine de bir şablon yazmamız gerekiyor. Görüşe bir `template_name` özneteliği ekleyerek hangi şablonun kullanılacağını açıkça belirtebiliriz, ancak açık bir şablon belirtilmediğinde Django, nesnenin adından bir şablon çıkarır. Bu durumda, çıkarılan şablon `"books/publisher_list.html"` olacaktır – `'books'` kısmı modeli tanımlayan uygulamanın adından gelirken, `"publisher"` kısmı modelin adının küçük harfli versiyonudur.

Not

Bu nedenle, örneğin `TEMPLATES` dosyasında bir `DjangoTemplates` arka ucunun `APP_DIRS` seçeneği `True` olarak ayarlandığında, şablonun konumu şu şekilde olabilir:
`/path/to/project/books/templates/books/publisher_list.html`

Bir şablon, tüm yayıncı nesnelerini içeren `object_list` adlı bir değişkeni barındıran bir bağlamda işlenecektir. Bir şablon şu şekilde görünebilir:

```
{% extends "base.html" %}

{% block content %}
    <h2>Yayıncılar</h2>
    <ul>
        {% for publisher in object_list %}
            <li>{{ publisher.name }}</li>
        {% endfor %}
    </ul>
{% endblock %}
```

Aslında hepsi bu kadar. Genel görünümün tüm kullanışlı özellikleri, genel görünüm üzerinde ayarlanan özelliklerin değiştirilmesinden kaynaklanmaktadır. Genel görünüm kılavuzu, tüm genel görünüm ve seçeneklerini ayrıntılı olarak açıklamaktadır; bu belgenin geri kalanında ise genel görünüm özelleştirmenin ve genişletmenin yaygın yöntemlerinden bazıları ele alınacaktır.

"Kullanıcı dostu" şablon bağlamları oluşturma

Örnek yayıncı listesi şablonumuzun tüm yayıncıları `object_list` adlı bir değişkende sakladığını fark etmiş olabilirsiniz. Bu, sorunsuz bir şekilde çalışsa da şablon yazarları için pek de "kullanıcı dostu" değildir: yazarların burada yayıncılarla uğraştıklarını "bilmeleri" gerekir.

Peki, bir model nesnesiyle uğraşıyorsanız, bu sizin için zaten yapılmıştır. Bir nesne veya sorgu kümesiyle uğraşırken, Django model sınıfının adının küçük harfli versiyonunu kullanarak bağlamı doldurabilir. Bu, varsayılan `object_list` girişine ek olarak sağlanır, ancak tam olarak aynı verileri içerir, yani `publisher_list`.

Bu da uygun değilse, bağlam değişkeninin adını manuel olarak belirleyebilirsiniz. Genel bir görünümdeki `context_object_name` özneteliği, kullanılacak bağlam değişkenini belirtir:

```
# views.py
from django.views.generic import ListView
from books.models import Publisher

class PublisherListView(ListView):
    model = Publisher
    context_object_name = "my_favorite_publishers"
```

Yararlı bir `context_object_name` belirtmek her zaman iyi bir fikirdir. Şablonları tasarlayan iş arkadaşlarınız size minnettar olacaktır.

Ek bilgi eklemek

Genellikle, genel görünümün sağladıklarının ötesinde bazı ek bilgiler sunmanız gerekir. Örneğin, her bir yayıncı ayrıntı sayfasında tüm kitapların bir listesini göstermeyi düşünün. `DetailView` genel görünümü, bağlama yayıncıyı sağlar; peki bu şablonda ek bilgileri nasıl elde edebiliriz?

Cevap, `DetailView`'un alt sınıfını oluşturmak ve `get_context_data` yönteminin kendi uygulamanızı sağlamaktır. Varsayılan uygulamaya, görüntülenen nesneyi şablona ekler, ancak daha fazlasını göndermek için bunu geçersiz kılabilirsiniz:

```
from django.views.generic import DetailView
from books.models import Book, Publisher

class PublisherDetailView(DetailView):
    model = Publisher

    def get_context_data(self, **kwargs):
```



```
# Önce temel uygulamayı çağır
context = super().get_context_data(**kwargs)
# Tüm kitapların QuerySet'ini ekle
context["book_list"] = Book.objects.all()
return context
```

Not

Genel olarak, `get_context_data` işlevi tüm üst sınıfların bağlam verilerini mevcut sınıfın verileriyle birleştirir. Bağlamı değiştirmek istediğiniz kendi sınıflarınızda bu davranışı korumak için, üst sınıfta `get_context_data` işlevini çağırduğunuzdan emin olmalısınız. Hiçbir sınıf aynı anahtarı tanımlamaya çalışmadığında, bu işlem beklenen sonuçları verecektir. Ancak, üst sınıflar bir anahtarı ayarladıktan sonra (super çağrısından sonra) herhangi bir sınıf bu anahtarı geçersiz kılmaya çalışırsa, o sınıfın tüm alt sınıflarının da tüm üst sınıfları geçersiz kılmak istiyorlarsa super çağrısından sonra bu anahtarı açıkça ayarlamaları gerekir. Sorun yaşıyorsanız, görünümünüzün yöntem çözümleme sırasını gözden geçirin.

Dikkate alınması gereken bir başka husus da sınıf tabanlı genel görünümlerden gelen bağlam verilerinin bağlam işleyicileri tarafından sağlanan verileri geçersiz kılmasıdır; örnek için `get_context_data()` işlevine bakın.

Nesnelerin alt kümelerini görüntüleme

Şimdi, başından beri kullandığımız model argümanına daha yakından bakalım. Görünümün üzerinde çalışacağı veri tabanı modelini belirten model argümanı, tek bir nesne veya nesneler koleksiyonu üzerinde çalışan tüm genel görünümelerde kullanılabilir. Ancak, görünümün üzerinde çalışacağı nesneleri belirtmenin tek yolu model argümanı değildir; nesnelerin listesini queryset argümanını kullanarak da belirtebilirsiniz:

```
from django.views.generic import DetailView
from books.models import Publisher

class PublisherDetailView(DetailView):
    context_object_name = "publisher"
    queryset = Publisher.objects.all()
```

`model = Publisher` ifadesini kullanmak, `queryset = Publisher.objects.all()` demenin kısaltmasıdır. Ancak, nesnelerin filtrelenmiş bir listesini tanımlamak için queryset kullanarak, görünümde görünecek nesneler konusunda daha spesifik olabilirsiniz (QuerySet nesneleri hakkında daha fazla bilgi için Sorgu Oluşturma bölümüne bakın; tüm ayrıntılar için sınıf tabanlı görünüm referansına bakın).

Bir örnek vermek gerekirse, kitap listesini yayın tarihine göre, en yenisi önce gelecek şekilde sıralamak isteyebiliriz:

```
from django.views.generic import ListView
from books.models import Book

class BookListView(ListView):
    queryset = Book.objects.order_by("-publication_date")
    context_object_name = "book_list"
```

Bu oldukça basit bir örnek, ancak konuyu gayet iyi açıklıyor. Genellikle nesneleri yeniden sıralamaktan daha fazlasını yapmak isteyeceksiniz. Belirli bir yayinevine ait kitapların listesini sunmak istiyorsanız, aynı tekniği kullanabilirsiniz:

```
from django.views.generic import ListView
from books.models import Book

class AcmeBookListView(ListView):
    context_object_name = "book_list"
    queryset = Book.objects.filter(publisher__name="ACME Publishing")
    template_name = "books/acme_list.html"
```

Filtrelenmiş bir sorgu kümesiyle birlikte özel bir şablon adı da kullandığımıza dikkat edin. Aksi takdirde, genel görünüm "vanilya" nesne listesiyle aynı şablonu kullanırdı ve bu bizim istediğimiz sonuç olmayabilirdi.

Ayrıca, bunun yayıncıya özgü kitapları oluşturmak için çok zarif bir yol olmadığını da unutmayın. Başka bir yayıncı sayfası eklemek istersek, URLconf'ta birkaç satır daha eklememiz gerekir ve yayıncı sayısı arttıkça bu durum mantıksız hale gelir. Bu sorunu bir sonraki bölümde ele alacağız.

Not

`/books/acme/` adresini çağırduğunuzda 404 hatası alırsanız, "ACME Publishing" adında bir Yayıncı'nın gerçekten mevcut olup olmadığını kontrol edin. Genel görünümde bu durum için bir `allow_empty` parametresi bulunur. Daha fazla ayrıntı için sınıf tabanlı görünüm kılavuzuna bakın.

Dinamik filtreleme

Bir başka yaygın ihtiyaç ise, liste sayfasında gösterilen nesneleri URL'deki bir anahtara göre filtrelemektir. Daha önce URLconf dosyasında yayıncının adını sabit olarak kodlamıştık; peki ya rastgele bir yayıncının tüm kitaplarını gösteren bir görünüm yazmak istersek ne yapacağız?

Neyse ki ListView sınıfında, yeniden tanımlayabileceğimiz bir `get_queryset()` yöntemi bulunmaktadır. Varsayılan olarak bu yöntem, `queryset` özneliğinin değerini döndürür; ancak biz bunu kullanarak daha fazla mantık ekleyebiliriz.

Bunun çalışmasını sağlayan en önemli nokta, sınıf tabanlı görünümler çağrıldığında, çeşitli yararlı bilgilerin `self` üzerinde depolanmasıdır; bu, istek (`self.request`) yanı sıra, URLconf'a göre yakalanan konumsal (`self.args`) ve ada dayalı (`self.kwargs`) argümanları da içerir.

Burada, tek bir yakalanmış gruba sahip bir URLconfumuz var:

```
# urls.py
from django.urls import path
from books.views import PublisherBookListView

urlpatterns = [
    path("books/<publisher>/", PublisherBookListView.as_view()),
]
```

Şimdi de `PublisherBookListView` görünümünü yazacağız:

```
# views.py
from django.shortcuts import get_object_or_404
from django.views.generic import ListView
from books.models import Book, Publisher

class PublisherBookListView(ListView):
    template_name = "books/books_by_publisher.html"

    def get_queryset(self):
        self.publisher = get_object_or_404(Publisher,
            name=self.kwargs["publisher"])
        return Book.objects.filter(publisher=self.publisher)
```

`get_queryset` işlevini kullanarak sorgu kümesi seçimine mantık eklemek hem kullanışlı hem de oldukça etkilidir. Örneğin, istersek `self.request.user` kullanarak mevcut kullanıcıya göre filtreleme yapabilir veya daha karmaşık mantıklar uygulayabiliriz.

Aynı zamanda yayıncıyı da bağlama ekleyebiliriz; böylece şablonda bunu kullanabiliriz:

```
# ...

def get_context_data(self, **kwargs):
    # Önce temel uygulamayı çağır
    context = super().get_context_data(**kwargs)
    # Yayıncıyı ekle
    context["publisher"] = self.publisher
    return context
```

Fazladan iş yapmak

İncelenecek son yaygın örüntü, genel görünümü çağırmadan önce veya sonra bazı ek işlemler yapmayı içerir.

Author modelimizde, birinin o yazara en son ne zaman baktığını takip etmek için kullandığımız bir `last_accessed` alanı olduğunu varsayalım:

```
# models.py
from django.db import models

class Author(models.Model):
    salutation = models.CharField(max_length=10)
    name = models.CharField(max_length=200)
    email = models.EmailField()
    headshot = models.ImageField(upload_to="author_headshots")
    last_accessed = models.DateTimeField()
```

Genel `DetailView` sınıfı bu alan hakkında hiçbir şey bilmez, ancak yine de bu alanı güncel tutmak için özel bir görünüm yazabiliriz.

Öncelikle, özel bir görünüme yönlendirmek üzere URLconf dosyasına bir yazar ayrıtısı bölümü eklememiz gerekir:

```
from django.urls import path
from books.views import AuthorDetailView

urlpatterns = [
    # ...
    path("authors/<int:pk>/", AuthorDetailView.as_view(), name="author-detail"),
]
```

Ardından yeni görünümümüzü yazacağız – `get_object`, nesneyi alan yöntemdir – bu yüzden onu geçersiz kılıp çağırıyor sarmalıyoruz:

```
from django.utils import timezone
from django.views.generic import DetailView
from books.models import Author

class AuthorDetailView(DetailView):
    queryset = Author.objects.all()

    def get_object(self):
        obj = super().get_object()
        # Son erişim tarihini kaydet
        obj.last_accessed = timezone.now()
        obj.save()
        return obj
```

Not

Buradaki URLconf, `pk` adlı grubu kullanır; bu isim, `DetailView`'un sorgu kümesini filtrelemek için kullanılan birincil anahtarın değerini bulmak üzere kullandığı varsayılan addır. Gruba başka bir ad vermek isterseniz, görünümde `pk_url_kwarg` ayarını yapabilirsiniz.

Sınıf tabanlı görünümle form işleme

Form işleme genel olarak 3 aşamadan oluşur:

- İlk GET isteği (boş veya önceden doldurulmuş form)
- Geçersiz verilerle POST isteği (genellikle hatalarla birlikte formun yeniden görüntülenmesi)
- Geçerli verilerle POST isteği (verilerin işlenmesi ve genellikle yönlendirme)

Bunu kendi başınıza uygulamak genellikle çok sayıda tekrarlanan standart kodla sonuçlanır (bkz. Görünümde form kullanma). Bunu önlemek için Django, form işleme amacıyla bir dizi genel sınıf tabanlı görünüm sunar.

Temel formlar

Aşağıdaki iletişim formu göz önüne alındığında:

```
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField()
    message = forms.CharField(widget=forms.Textarea)

    def send_email(self):
        # self.cleaned_data sözlüğünü kullanarak e-posta gönder
        pass
```

Görünüm, bir `FormView` kullanılarak oluşturulabilir:

```
from myapp.forms import ContactForm
from django.views.generic.edit import FormView

class ContactFormView(FormView):
    template_name = "contact.html"
    form_class = ContactForm
    success_url = "/thanks/"

    def form_valid(self, form):
```

```
# Bu yöntem geçerli form verisi POST edildiğinde çağrılır.
# HttpResponse döndürmelidir.
form.send_email()
return super().form_valid(form)
```

Notlar:

- FormView, TemplateResponseMixin sınıfını devralır; bu sayede burada template_name kullanılabilir.
- form_valid() işlevinin varsayılan uygulaması, sadece success_url adresine yönlendirme yapar.

Örnek formlar

Genel görünüm, modellerle çalışırken gerçekten çok kullanışlıdır. Bu genel görünüm, hangi model sınıfının kullanılacağını belirleyebildikleri sürece otomatik olarak bir ModelForm oluşturur:

- model özneliği belirtilmişse, o model sınıfı kullanılır.
- get_object() bir nesne döndürürse, o nesnenin sınıfı kullanılır.
- bir sorgu kümesi belirtilmişse, o sorgu kümesinin modeli kullanılır.

Model form görünümü, modeli otomatik olarak kaydeden bir form_valid() uygulaması sunar. Özel gereksinimleriniz varsa bunu geçersiz kılabilirsiniz; örnekler için aşağıya bakın.

CreateView veya UpdateView için bir success_url belirtmenize bile gerek yoktur; bunlar, varsa model nesnesinde get_absolute_url() işlevini kullanır.

Özel bir ModelForm kullanmak istiyorsanız (örneğin, ek doğrulama eklemek için), görünümünüzde form_class ayarını yapın.

Not

Özel bir form sınıfı belirlerken, form_class bir ModelForm olsa bile yine de modeli belirtmeniz gerekir.

Öncelikle Author sınıfımıza get_absolute_url() işlevini eklememiz gerekiyor:

```
from django.db import models
from django.urls import reverse

class Author(models.Model):
    name = models.CharField(max_length=200)

    def get_absolute_url(self):
        return reverse("author-detail", kwargs={"pk": self.pk})
```

Ardından, asıl işi yapmak için CreateView ve benzer yöntemlerini kullanabiliriz. Burada sadece genel sınıf tabanlı görünümü yapılandırdığımıza dikkat edin; herhangi bir mantık kodunu kendimiz yazmamız gerekmiyor:

```
from django.urls import reverse_lazy
from django.views.generic.edit import CreateView, DeleteView, UpdateView
from myapp.models import Author

class AuthorCreateView(CreateView):
    model = Author
    fields = ["name"]

class AuthorUpdateView(UpdateView):
    model = Author
    fields = ["name"]

class AuthorDeleteView(DeleteView):
    model = Author
    success_url = reverse_lazy("author-list")
```

Not

Dosya içe aktarıldığında URL'ler yüklenmediği için, reverse() yerine reverse_lazy() işlevini kullanmamız gerekiyor.

fields özniteliği, ModelForm'daki iç Meta sınıfındaki fields özniteliği ile aynı şekilde çalışır. Form sınıfını başka bir şekilde tanımladığınızın sürece, bu öznitelik zorunludur ve eksik olması durumunda görünüm bir ImproperlyConfigured istisnası oluşturur.

Hem fields hem de form_class özniteliklerini belirtirseniz, bir ImproperlyConfigured istisnası oluşturulur.

Son olarak, bu yeni görünümleri URLconf'a ekliyoruz:

```
from django.urls import path
from myapp.views import AuthorCreateView, AuthorDeleteView, AuthorUpdateView

urlpatterns = [
    # ...
    path("author/add/", AuthorCreateView.as_view(), name="author-add"),
    path("author/<int:pk>/", AuthorUpdateView.as_view(), name="author-update"),
    path("author/<int:pk>/delete/", AuthorDeleteView.as_view(), name="author-delete"),
]
```

Not

Bu görünüm, modele göre template_name öğesini oluşturmak için template_name_suffix öğesini kullanan SingleObjectTemplateResponseMixin sınıfını devralır.

Bu örnekte:

- CreateView ve UpdateView, myapp/author_form.html dosyasını kullanır
- DeleteView, myapp/author_confirm_delete.html dosyasını kullanır

CreateView ve UpdateView için ayrı şablonlar kullanmak isterseniz, görünüm sınıfınızda template_name veya template_name_suffix ayarlarından birini belirleyebilirsiniz.

Modeller ve request.user

CreateView kullanarak bir nesne oluşturan kullanıcıyı takip etmek için, bunu yapmak üzere özel bir ModelForm kullanabilirsiniz. Öncelikle, modele yabancı anahtar ilişkisini ekleyin:

```
from django.contrib.auth.models import User
from django.db import models

class Author(models.Model):
    name = models.CharField(max_length=200)
    created_by = models.ForeignKey(User, on_delete=models.CASCADE)

    # ...
```

Görünümde, düzenlenecek alanlar listesine created_by alanını eklemediğinizden emin olun ve kullanıcıyı eklemek için form_valid() işlevini yeniden tanımlayın:

```
from django.contrib.auth.mixins import LoginRequiredMixin
from django.views.generic.edit import CreateView
from myapp.models import Author

class AuthorCreateView(LoginRequiredMixin, CreateView):
    model = Author
    fields = ["name"]

    def form_valid(self, form):
        form.instance.created_by = self.request.user
        return super().form_valid(form)
```

LoginRequiredMixin, oturum açmamış kullanıcıların forma erişmesini engeller. Bunu kullanmazsanız, yetkisiz kullanıcılar form_valid() işlevinde yönetmeniz gerekir.

İçerik uyumu örneği

Aşağıda hem API tabanlı iş akışlarıyla hem de "normal" form POST'larıyla çalışan bir formu nasıl uygulayabileceğinizi gösteren bir örnek yer almaktadır:

```
from django.http import JsonResponse
from django.views.generic.edit import CreateView
from myapp.models import Author
```

```
class JsonableResponseMixin:
    """
    Bir forma JSON desteği eklemek için Mixin.
    Nesne tabanlı FormView ile kullanılmalıdır (örn. CreateView)
    """

    def form_invalid(self, form):
        response = super().form_invalid(form)
        if self.request.accepts("text/html"):
            return response
        else:
            return JsonResponse(form.errors, status=400)

    def form_valid(self, form):
        response = super().form_valid(form)
        if self.request.accepts("text/html"):
            return response
        else:
            data = {
                "pk": self.object.pk,
            }
            return JsonResponse(data)

class AuthorCreateView(JsonableResponseMixin, CreateView):
    model = Author
    fields = ["name"]
```

Yukarıdaki örnekte, istemcinin text/html formatını desteklemesi durumunda bunu tercih edeceği varsayılmaktadır. Ancak bu her zaman geçerli olmayabilir. Bir .css dosyası talep edildiğinde, birçok tarayıcı `Accept: text/css, */*;q=0.1` başlığını gönderir; bu da CSS'yi tercih ettiklerini, ancak başka formatların da kabul edilebilir olduğunu belirtir. Bu durumda `request.accepts("text/html")` değeri True olacaktır.

İstemcinin tercihini dikkate alarak doğru biçimi belirlemek için `django.http.HttpRequest.get_preferred_type()` işlevini kullanın:

```
class JsonableResponseMixin:
    """
    Bir forma JSON desteği eklemek için Mixin.
    Nesne tabanlı FormView ile kullanılmalıdır (örn. CreateView).
    """

    accepted_media_types = ["text/html", "application/json"]

    def dispatch(self, request, *args, **kwargs):
        if request.get_preferred_type(self.accepted_media_types) is None:
            # Ortak format yok.
            return HttpResponse(
                status_code=406, headers={"Accept":
                    ",".join(self.accepted_media_types)}
            )

        return super().dispatch(request, *args, **kwargs)

    def form_invalid(self, form):
        response = super().form_invalid(form)
        accepted_type = self.request.get_preferred_type(self.accepted_media_types)
        if accepted_type == "text/html":
            return response
        elif accepted_type == "application/json":
            return JsonResponse(form.errors, status=400)

    def form_valid(self, form):
        response = super().form_valid(form)
        accepted_type = self.request.get_preferred_type(self.accepted_media_types)
        if accepted_type == "text/html":
            return response
        elif accepted_type == "application/json":
            data = {
                "pk": self.object.pk,
            }
```

```
return JsonResponse(data)
```

Django 5.2'de yapılan değişiklik: `HttpRequest.get_preferred_type()` yöntemi eklendi.